

NSFlow: An End-to-End FPGA Framework with Scalable Dataflow Architecture for Neuro-Symbolic AI

Hanchen Yang^{*1}, Zishen Wan^{*1}, Ritik Raj¹, Joongun Park¹, Ziwei Li¹, Ananda Samajdar²,
Arijit Raychowdhury¹, Tushar Krishna¹

¹Georgia Institute of Technology, Atlanta, GA ²IBM Research, Yorktown Heights, NY

Abstract—Neuro-Symbolic AI (NSAI) is an emerging paradigm that integrates neural networks with symbolic reasoning to enhance the transparency, reasoning capabilities, and data efficiency of AI systems. Recent NSAI systems have gained traction due to their exceptional performance in reasoning tasks and human-AI collaborative scenarios. Despite these algorithmic advancements, executing NSAI tasks on existing hardware (e.g., CPUs, GPUs, TPUs) remains challenging, due to their heterogeneous computing kernels, high memory intensity, and unique memory access patterns. Moreover, current NSAI algorithms exhibit significant variation in operation types and scales, making them incompatible with existing ML accelerators. These challenges highlight the need for a versatile and flexible acceleration framework tailored to NSAI workloads.

In this paper, we propose NSFlow, an FPGA-based acceleration framework designed to achieve high efficiency, scalability, and versatility across NSAI systems. NSFlow features a design architecture generator that identifies workload data dependencies and creates optimized dataflow architectures, as well as a reconfigurable array with flexible compute units, re-organizable memory, and mixed-precision capabilities. Evaluating across NSAI workloads, NSFlow achieves $31\times$ speedup over Jetson TX2, more than $2\times$ over GPU, $8\times$ speedup over TPU-like systolic array, and more than $3\times$ over Xilinx DPU. NSFlow also demonstrates enhanced scalability, with only $4\times$ runtime increase when symbolic workloads scale by $150\times$. To the best of our knowledge, NSFlow is the first framework to enable real-time generalizable NSAI algorithms acceleration, demonstrating a promising solution for next-generation cognitive systems.

I. INTRODUCTION

Neuro-Symbolic AI (NSAI) emerges as a promising paradigm toward achieving artificial general intelligence (AGI) and human-like fluid intelligence. Compared to deep neural networks (DNNs), NSAI exhibits superior performance in cognitive tasks such as human-like learning, reasoning, and logical thinking [1]–[9]. NSAI synergistically combines neural approaches (e.g., DNNs) with symbolic representations (e.g., vectors, logics, graphs) to advance cognitive ability [10]–[16].

Despite its cognitive advantages, achieving real-time and efficient NSAI inference on resource-constrained devices presents significant challenges. These challenges stem from higher memory intensity, greater computational kernel heterogeneity, irregular memory access patterns, and underutilization of hardware resources. In our experiments, it takes >3 mins on NVIDIA desktop GPU to perform single reasoning task [17], underscoring the inefficiency of current solutions.

Previous work has identified three main challenges of NSAI [18]: *First, high memory footprint.* NSAI systems heavily rely on vector-symbolic architectures (VSAs) that use vector operations to encode symbolic knowledge, resulting in large memory footprints (often tens to hundreds of MB) and making it impractical to be fully cached on-chip in hardware accelerators. *Second, heterogeneous compute kernels.* Beyond neural networks, NSAI workloads incorporate diverse computations of varying sizes, such as vector convolutions, element-wise operations, and logical reasoning. These exhibit low data reuse,

low compute array utilization, and limited parallelism, leading to inefficiencies on GPUs and TPUs. *Third, critical path dependency.* Symbolic reasoning often depends on outputs from neuro-perceptual modules, extending the critical path during cognitive inference and causing underutilization of traditional accelerators.

With the growing demand for scalable dataflow and architecture solutions for NSAI, FPGAs present an ideal platform due to their customizability, flexible memory management, and reconfigurability to adapt to evolving NSAI workloads. Previous work has demonstrated the potential of FPGAs for accelerating ML workloads [19]–[24]. However, FPGA deployment remains challenging for NSAI algorithms due to the complexity of organizing on-chip resources and limited memory capacity [25]–[27].

To address these challenges, we identify unique opportunities to enhance NSAI acceleration efficiency and propose NSFlow, a scalable FPGA-based dataflow architecture design automation framework. To the best of our knowledge, NSFlow is the *first* automated end-to-end solution for accelerating and deploying generic NSAI workloads. NSFlow features a frontend subsystem with *dataflow architecture generator* that includes NSAI execution trace extraction, dataflow graph generation, and a two-phase design space co-exploration strategy, and a backend subsystem with *flexible neuro-symbolic hardware architecture* with adaptive array folding, reconfigurable memory partitioning, and efficient heterogeneous storage. By integrating its frontend and backend, NSFlow delivers efficient and scalable acceleration for NSAI workloads. Specifically, it identifies data dependencies, explores design space options, and generates optimized dataflow architectures tailored for FPGA deployment.

This paper, therefore, makes the following contributions:

- 1) An end-to-end FPGA design automation framework for accelerating and deploying generic NSAI workloads.
- 2) A design generator that (i) identifies workload-specific data dependencies using a self-generated dependency graph tailored for vector-symbolic-based NSAI algorithms and (ii) derives an optimal dataflow architecture through a novel design space co-exploration strategy.
- 3) A hardware architecture featuring a flexible neuro-symbolic systolic array, an efficient SIMD unit, reorganizable on-chip memory, and support for mixed-precision computations.

II. NEURO-SYMBOLIC AI AND CHARACTERIZATION

This section presents NSAI algorithms with key kernels (Sec. II-A), and analyzes their workload characteristics (Sec. II-B).

A. Neuro-Symbolic AI Algorithm

Neurosymbolic AI synergistically integrates *learning capability* of neural networks with *reasoning capability* of symbolic AI, offering data-efficient learning and transparent, logical decision-making beyond traditional DNNs. **1) Neural system.** The process begins with a neural module that handles perception tasks by interpreting sensory data and generating meaningful scene and object representations,

^{*}Equal Contributions. (hanchen@gatech.edu, zishenwan@gatech.edu)

This work was supported in part by CoCoSys, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation program sponsored by DARPA.

TABLE I

NEUROSymbolic MODELS. SELECTED NEUROSymbolic AI WORKLOADS FOR ANALYSIS, REPRESENTING A DIVERSE OF APPLICATION SCENARIOS.

Representative Neuro-Symbolic AI Workloads	Neuro-Vector-Symbolic Architecture (NVSA) [17]	Multiple-Input-Multiple-Output Neural Networks (MIMONet) [28]	Probabilistic Abduction via Learning Rules in Vector-symbolic Architecture (LVRF) [12]	Probabilistic Abduction and Execution Learner (PrAE) [5]
Compute Pattern	Neuro	CNN	CNN/Transformer	CNN
Use Case	Symbolic	VSA binding/unbinding (Circular Conv)	VSA binding (Circular Conv)	VSA binding/unbinding (Circular Conv)
Advantage vs. Neural	Spatial-temporal and abstract reasoning	Multi-input simultaneously processing	Probabilistic reasoning, OOD data processing	Probabilistic abduction
Application Scenario	Higher joint representation efficiency, Better reasoning capability, Transparency	Higher throughput, Lower latency, Compositional compute, Transparency	Stronger OOD handling capability, One-pass learning, Higher flexibility, Transparency	Spatial-temporal and abstract reasoning
				Higher generalization, Transparency, Interpretability, Robustness

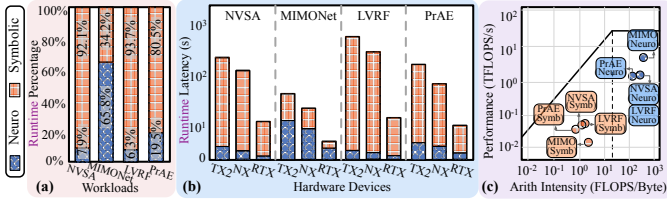
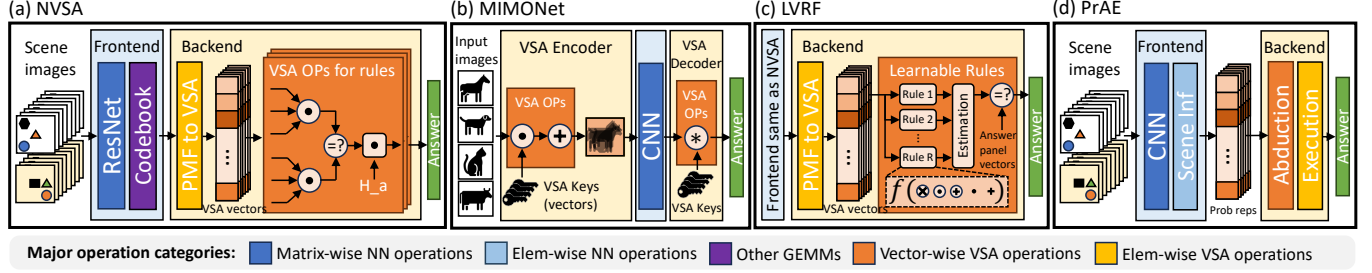


Fig. 1. **End-to-end neuro-symbolic runtime and roofline characterization.** (a) Benchmark neuro-symbolic models on CPU+GPU system, showing symbolic may serve as system bottleneck. (b) Benchmark on Coral TPU, TX2, NX, and 2080Ti GPU, showing that real-time performance cannot be satisfied. (c) Roofline of RTX 2080Ti GPU, indicating symbolic memory-bound.

providing essential inputs for reasoning. ② **Symbolic system.** These features are then passed to the symbolic system for reasoning, enhancing explainability and reducing reliance on extensive training data by leveraging established models of the physical world (e.g., rules and coded knowledge). This step integrates learned neural network knowledge with symbolic rules, allowing the system to both learn from new data and reason logically based on existing knowledge. The outputs of symbolic reasoning are used for decision-making, and response or action generation.

Tab. I highlights four representative neuro-symbolic workloads: NVSA [17] for spatial-temporal reasoning, MIMONet for multi-input processing [28], LVRF for probabilistic abduction [12], and PrAE for abstract reasoning tasks [5]. These workloads demonstrate superior reasoning capabilities and represent a promising paradigm for human-like intelligence. These approaches integrate CNNs for neuro and vector-symbolic architectures (VSAs) for symbolic processing.

A key VSA operation is the blockwise *circular convolution* that combines two vectors in a way that preserves the information from both, making it suitable for representing composite symbols. Mathematically, the circular convolution of two vectors **A** and **B** (each of dimension N) generates vector **C** as $C[n] = \sum_{k=0}^{N-1} A[k] \cdot B[(n-k) \bmod N]$ where each element of **C** is obtained by multiplying the elements of **A** with the circularly shifted elements of **B**, and then summing up. Circular convolution has commutativity and associativity properties, making it particularly effective in hierarchical reasoning tasks where manipulating structured information is critical.

B. Neuro-Symbolic AI Workload Characterization

To understand the real-device efficiency of neuro-symbolic AI workload, recent work [29] profiles four representative models as

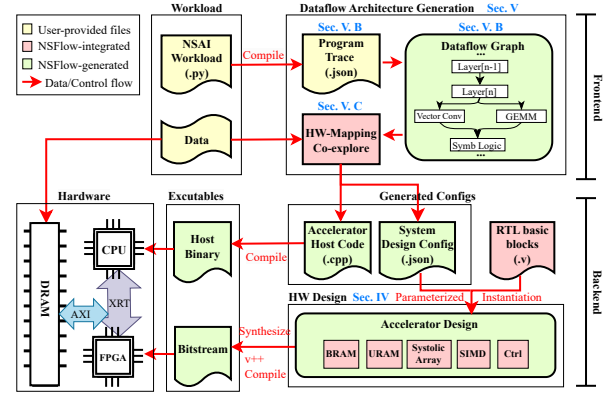


Fig. 2. **NSFlow Overview.**

elaborated in Tab. I on Coral edge TPU (4 W), Jetson TX2 (15 W), Xavier NX (20 W), and RTX 2080Ti (250 W), respectively.

End-to-end latency breakdown. Fig. 1a and Fig. 1b present the end-to-end latency breakdown of neuro-symbolic workloads, highlighting three key observations: (1) *The real-time performance cannot be satisfied* across devices. Even with additional compute resources to reduce NN runtime, the substantial overhead from symbolic reasoning prevents real-time execution. (2) *Symbolic operations dominate runtime.* For instance, symbolic modules account for 87% of NVSA total runtime while contributing only 19% of its total FLOPS, suggesting that symbolic operations are not efficiently handled by GPUs/TPUs. (3) *Symbolic reasoning computation lies on the critical path* as its computation depends on outputs from the neural modules.

System Roofline Analysis. Fig. 1c employs the roofline model of RTX 2080Ti GPU version to quantify the neurosymbolic workloads. We observe that *symbolic modules are memory-bound while neuro modules are compute-bound*. This is mainly due to symbolic operations requiring streaming vector elements, increasing the memory bandwidth pressure and resulting in hardware underutilization.

III. NSFLOW OVERVIEW

NSFlow is an end-to-end framework that identifies data dependency, explores the design space, and generates an optimal dataflow architecture design for FPGA deployment tailored to a given NSAI workload. Fig. 2 (a) shows an overview of the proposed framework, divided into frontend and backend.

A. NSFlow Frontend

The *Design Architecture Generator (DAG)* is the core component of the frontend operating on the host side. The DAG module begins by extracting an execution trace from the user-provided workload. It then generates a dataflow graph specifically designed for VSA-based NSAI workloads, capturing operator-level specifications, runtime, memory functions, and their data dependencies. This dataflow graph is used for co-exploration of dataflow architecture through a novel two-phase algorithm. The first phase identifies the optimal system design configuration for the FPGA accelerator, while the second determines an efficient (or near-optimal) reconfiguration and mapping scheme. These configurations are specified in the accelerator host code, enabling the CPU to invoke device kernels via the XRT API. After compilation, the CPU executes the host binary to schedule operations on the FPGA.

B. NSFlow Backend

The NSFlow backend includes a pre-define accelerator template comprising several essential components: BRAM blocks for flexible on-chip memory, adaptive Systolic Array for parallel neuro and symbolic operations, SIMD unit for element-wise, vector reduction and scalar operations, and control logic for task scheduling on hardware level. These components are parameterized using the system design configuration file generated by the frontend, enabling the instantiation of an optimized microarchitecture based on workload characterization. NSFlow then synthesizes and compiles the RTL into an executable bitstream for deployment. During real-time inference, the CPU executes the host binary code to run FPGA kernels and manages off-chip memory transactions through AXI interfaces.

We will present the NSFlow design in a bottom-up manner, starting with the backend flexible neuro-symbolic hardware architecture (Sec. IV) and followed by the frontend graph and dataflow architecture generators (Sec. V).

IV. NSFLOW BACKEND: FLEXIBLE HARDWARE ARCHITECTURE

This section first presents an overview of the NSFlow hardware architecture (Sec. IV-A), then walks through our design featuring an *NS-adaptive systolic array* (Sec. IV-B), *Re-organizable on-chip memory* (Sec. IV-C), *Adaptive mixed precision computation* (Sec. IV-D), an *Efficient custom SIMD unit* (Sec. IV-E).

A. Overview of NSFlow Hardware Architecture

Fig. 3(a) exhibits the hardware architecture of NSFlow. It consists of a uniquely designed NSAI-workload-adaptive Systolic Array for NN and Vector-symbolic operations, a SIMD unit for reductions, element-wise operations, flexibly arranged on-chip RAM blocks, and a control unit for kernel scheduling and memory transactions. NSFlow has pre-defined RTL of all the above blocks with scaling parameters subject to the design configuration generated from DAG for optimal execution.

B. Adaptive Systolic Array (AdArray)

Inspired by [29], we implement an adaptive systolic array design (AdArray) to maximize efficiency for NSAI inference.

Adaptive array folding. AdArray can run both NN ops and vector-symbolic circular convolution, the two most dominating components in our targeted NSAI workload, on its arbitrary portions (sub-arrays) simultaneously to maximize parallelism and utilization. Each sub-array is either combined with its adjacent one to operate NN ops, or singularly running vector operations like circular convolution in the symbolic binding process. In Fig. 3(a) we showcase the design with

a mini 4×6 systolic array, which is split into 3 sub-arrays with each ranging 2 columns (c0 and c1 for A1, c2 and c3 for A2, c4 and c5 for A3). In this case, A1 and A2 are combined together to perform NN ops, while in A3 each column is running vector-symbolic operations.

Efficient vector-symbolic circular convolution streaming. Traditional TPU's systolic array is extremely inefficient for circular convolution operations with heavy memory transactions and low parallelism due to non-ideal spatial and temporal mapping. Fig. 3(b) showcases how a single column in our design performs vector-symbolic operation with a 3-element circular convolution example. The first vector A is held in stationary registers, while the second vector B is streamed from SRAM. The MAC unit processes the data from both stationary and streaming registers, adding it to the partial product received from the PE above. A passing register temporarily stores the streaming input for a cycle before it moves to the streaming register. This value is transferred to the passing register of the next PE in the following cycle. The procedure is repeated until the final circular convolution outputs. Unlike traditional Systolic Arrays, each PE uses an extra register named Passing Reg at the top of one of the input ports to cause a 1-cycle streaming pace mismatch between the two input vectors A and B , thus enabling circular convolution operations. Note that to enable this type of streaming, each PE needs to have one extra vertical input port connected with its above PE's previous right output port as depicted in Fig. 3(b). When performing NN operations, the Passing Register is bypassed via multiplexer, and the horizontal connections in the sub-array are again established to enable weight and input passing as in a traditional Systolic Array.

Two-level flexibility. Our efficient systolic array design also benefits from extraordinary flexibility at both design level and kernel level. At design level, our DAG decides the array size and its memory size (Sec. IV-C), as well as the number of sub-arrays that best fits the overall workload characteristic (Sec. V); At kernel level, the array are reconfigured to an optimal folding scheme at runtime, maximizing utilization and parallelism for the NN and vector-symbolic operations.

C. Re-Organizable On-Chip Memory

The profiled NSAI workloads feature heavy memory usage and versatile computing kernels, thus we design a flexible memory system to enable smooth executions and transactions with limited FPGA on-chip memory resource ($\sim 36\text{MB}$ on ZCU104), which features **1 Re-organizable memory partition**, **2 Adaptive memory size**, and **3 Efficient heterogeneous storage** to fully exploit FPGA's potential and maximize memory efficiency for NSAI workloads.

As shown in Fig. 3, the on-chip memory system consists of three memory blocks Mem_A , Mem_B , and Mem_C , an on-chip cache, and a memory bus for off-chip transactions. Mem_A , Mem_B , and Mem_C are all double-buffered memories to enable seamless read and write among off-chip memory and the systolic array. **1** Mem_A is partitioned into two chunks - Mem_{A1} and Mem_{A2} - to simultaneously load NN layers and vector data for the corresponding sub-array in AdArray. When performing NN operations or vector operations singularly, the two memory chunks can be merged into one at runtime for better performance and simpler control. Mem_B works as the IFMAP buffer in normal systolic arrays which feeds data to the horizontal inputs of AdArray only for NN processing. Mem_C stores the outputs from AdArray and the SIMD unit which are either read by the compute units, or written to Mem_A/Mem_B or off-chip DRAM. The on-chip cache buffers intermediate results for the 3 memory blocks. **2** The sizes of all above memory components will be defined by DAG based on workload's characteristics and dataflow. **3** In real FPGA deployment, Mem_A , Mem_B , and Mem_C are comprised of

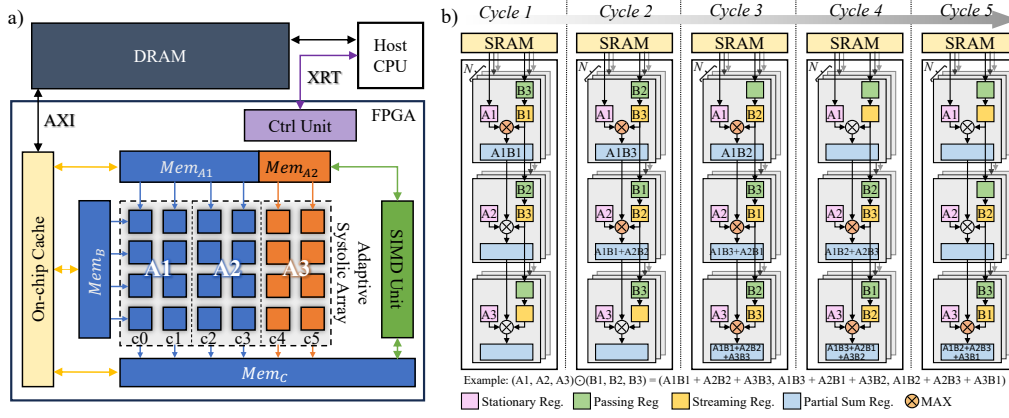


Fig. 3. NSFlow Hardware Architecture.

numerous 18KB BRAM blocks for maximum configurability, and on-chip cache is built with URAM considering its large capacity (288KB per block). Small registers and buffers in compute element use LUTRAMs for fast and dynamic access.

D. Adaptive Compute for Mixed Precision

To improve computing efficiency and save on-chip memory usage, NSFlow supports mixed precisions ranging from FP16/8 to INT8/4 in different components of the workload specified by user at frontend. DAG employs compute units adaptive to various precisions. In NVSA for example, NN and Symbolic operations are quantized to INT8 and INT4 respectively, thus the multipliers in AdArray and the SIMD support both precisions with sufficient leverage of DSP units [30]. Low-precision additions are handled by LUT for fast outcome.

E. Efficient Custom SIMD Unit

NSFlow incorporates a custom SIMD unit to efficiently perform vector reductions, element-wise operations, etc., with fluid data transfer between the output of the NSFlow array and the input SRAM for successive executions. It comprises multiple processing elements (PEs), each equipped with compact logic circuits (i.e., sum, mult/div, exp/log/tanh, norm, softmax, etc.) to handle vector operations or optimized sparse computations on mixed level of quantized data.

V. NSFLOW FRONTEND: DATAFLOW ARCHITECTURE GENERATION

In the frontend, we implement a *Dataflow Architecture Generator (DAG)* that first builds a dataflow graph based on operation trace extracted from the workload, then generates an optimal (or sub-optimal) dataflow architecture design, defined by a design configuration file for instantiating hardware modules, and a host code for the CPU to schedule accelerator kernels.

This section first identifies the NSAI dataflow challenges (Sec. V-A). Then we illustrates the process to generate *operation graph* and subsequently *dataflow graph* (Sec. V-B). Finally, we discuss how *DAG* searches for an optimal architectural design and mapping based on the *dataflow graph* (Sec. V-C).

A. NSAI Dataflow Challenges

We identify three main NSAI dataflow challenges (Fig. 4(b)). First, the sequential execution and frequent interactions of neural and symbolic components results in increased latency and low system throughput. Second, the heterogeneous neural and symbolic kernels lead to low compute array utilization and efficiency of ML accelerators. Third, heavy memory transactions exhibited in both

components can cause large communication latency, which is even more challenging in FPGA deployment.

DAG perfectly addresses the above challenges by first, identifying data dependencies through dataflow graph to fully exploit parallelism opportunities among NN and symbolic operations with featured HW architecture; second, balancing NN and symbolic operations on our AdArray, empowered by both *design-level flexibility* and *kernel-level flexibility* to achieve maximum utilization; third, configuring memory units adaptively to best-fit workload's memory usage, thus eliminating unnecessary transactions and stalls.

B. Data Dependency Identification

```
graph():
...
// Neuro Operation - CNN (Resnet18)
%relu_1[16,64,160,160] : call_module[relu](args = (%bn1
[16,64,160,160]))
%maxpool_1[16,64,160,160] : call_module[maxpool](args =
(%relu_1[16,64,160,160]))
%conv2d_1[16,64,160,160] : call_module[conv2d](args =
(%maxpool_1[16,64,160,160]))
...
// Symbolic Operations
// Inverse binding of two block codes vectors by
blockwise circular correlation
%inv_binding_circular_1[1,4,256] : call_function[nvsa.
inv_binding_circular](args = (%vec_0[1,4,256], %
vec_1[1,4,256]))
%inv_binding_circular_2[1,4,256] : call_function[nvsa.
inv_binding_circular](args = (%vec_3[1,4,256], %
vec_4[1,4,256]))
// Compute similarity between two block codes vectors
%match_prob_1[1] : call_function[nvsa.match_prob](args
= (%inv_binding_circular_1[1,4,256], %vec_2
[1,4,256]))
// Compute similarity between a dictionary and a batch
of query vectors
%match_prob_multi_batched_1[1] : call_function[nvsa.
match_prob_multi_batched](args = (%
inv_binding_circular_2[1,4,256], %vec_5[7,4,256]))
%sum_1[1] : call_function[torch.sum](args = (%
match_prob_multi_batched_1[1]))
%clamp_1[1] : call_function[torch.clamp](args = (%sum_1
[1]))
%mul_1[1] : call_function[operator.mul](args = (%
match_prob_1[1], %clamp_1[1]))
...
```

Listing 1. Neuro-Vector-Symbolic Architecture Profiling Result

Program trace. NSFlow first extracts an execution trace from input program through compilation, and pre-process the file to be ready for later dataflow graph generating. Listing 1 exhibits a snapshot taken from NVSA program trace, with representative kernels for Neural and Symbolic parts showing data dependencies.

Dataflow Graph. Fig. 4 depicts how a *Dataflow Graph* is derived.

❶ **Critical path identification:** It begins with a DFS through the execution graph previously generated, identifying the critical

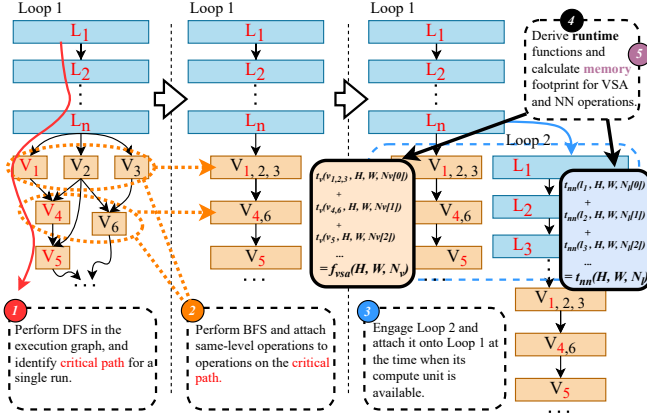


Fig. 4. Dataflow Architecture Generation (DAG) Flow.

TABLE II

NSFLOW DESIGN SPACE. Maximum #PEs = 2^m . With *exploration phasing* and *space pruning*, search space is reduced by 100 magnitudes

	HW config (H, W, N)	Array partition and mapping	Total design space, $m = 10$
Original	$m \times (m + 1)/2$	$(N - 1)^k$ for each N	10^{300}
DAG	Phase I: $1/4 \leq H/W \leq 16$	Phase II: $\text{Iter} \times \text{\#layers}$	10^3

path for a single loop of the workload. **② Inner-loop parallelism identification:** Then DAG walks through the graph again with BFS, to identify operation nodes at the same depth as the nodes on the critical path, and attach them to the corresponding critical-path nodes, indicating their earliest execution and parallelisms. For a single loop, NN layers are typically on the critical path without any attached nodes due to its strict dependencies between layers, while symbolic parts may have more parallelism opportunities thus more grouped nodes. **③ Inter-loop parallelism identification:** After reshaping the graph for a single loop, DAG attaches the next loop's graph to the existing one by positioning the first operation of the second loop at the time its required computing unit is freed. For example in the third step shown in Fig. 4(b), the first NN layer (L_1) in Loop 2 starts as soon as the last NN layer (L_n) of Loop 1 finishes and runs along with the symbolic operations in Loop 1. **④ Runtime function derivation:** For each node in the newly fused graph, DAG derives their runtime functions (Sec. V-C) on corresponding sub-arrays with operation parameters (i.e. vector quantity n and dimension d , NN layer dimensions in m, n, k , etc.) and configuration variables (i.e. sub-array's height, width and partition scheme). **⑤ Memory cost calculation:** DAG also computes memory footprint based on each node's data size for later memory block configuring.

Dataflow Graph describes data dependencies, inner/inter-loop parallelisms as well as their runtime and memory cost model for real-time execution on AdArray. Next we discuss how DAG uses it to explore the HW design and mapping.

C. Two-Phase Design Space Exploration

Design Space. NSFlow's adaptive architecture (Sec. IV) and the Dataflow Graph (Sec. V-B) creates a large cross-coupled design space, defined by the hardware configuration with height (H), width (W) and number (N) of the sub-arrays, and the mapping scheme specified by the number of sub-arrays running NN layers and VSA operations (i.e. $N_l[i]$ for layer node i and $N_v[j]$ for VSA node j , where j could vary for each node in the Dataflow Graph during runtime. Note that N_l and N_v are both vector variables with lengths equal to the number of layer and VSA nodes in a single loop. The total size of this design space reaches 10^{300} (Tab. IV), making brutal force search impractical. Next we present how we derive runtime function for the nodes and our unique DSE algorithm.

Algorithm 1: NSFlow Two-Phase DSE Algorithm

Data: $R_l, R_v, Range_H$ (H search range), $Range_W$ (W search range), M (max #PEs), $Iter_{max}$ (Phase II max iterations)
Result: H, W, N (total #sub-arrays), N_l, N_v

```

1 /* Phase I */
2 for  $H$  in  $Range_H, W$  in  $Range_W$  do
3    $N = \lfloor M / (H \times W) \rfloor$  // get total #sub-arrays
4   for  $\bar{N}_l$  in  $[1, N]$  do
5     // get optimal HW config for parallel mapping
6     Set all elements in  $N_l$  to  $\bar{N}_l$ 
7     Set all elements in  $N_v$  to  $N - \bar{N}_l$ 
8      $t_{para} = \max(t_{nn}(H, W, N_l), t_{vsa}(H, W, N_v))$ 
9     Save the  $H, W, \bar{N}_l$  (and  $\bar{N}_v$ ) with minimal  $t_{para}$ .
10  end
11 // get sequential runtime
12  $t_{seq} = \sum f_i(H, W, N) +$ 
13    $\min(\sum f_{v,j,temp}(H, W, N), \sum f_{v,j,spatial}(H, W, N))$ 
14 // Set to sequential mode in case it has better performance
15 Return and set sequential mode if  $t_{seq} < t_{para}$  else Continue
16 /* Phase II */
17 for  $it$  in  $Iter_{max}$  do
18   for layer  $i$  in  $R_l$  do
19     Locate VSA node  $j'$  and  $j''$  where layer  $i$  starts and ends
20     if  $t_{seq} < t_{para}$  do  $N_l[i] --;$ ;  $N_v[j' : j''] ++;$ 
21     else do  $N_l[i] ++;$ ;  $N_v[j' : j''] --;$ 
22      $t_{para} = \max(t_{nn}(H, W, N_l), t_{vsa}(H, W, N_v))$ 
23     Save the  $H, W, N_l, N_v$  with minimal  $t_{para}$ .
24   end
25 end
26 Return  $H, W, N, N_l, N_v$ .

```

Analytical models. Inspired by the analytical models from previous research [29], [31], we derive runtime functions specifically for NSFlow. Since AdArray is a scale-out design with row-level partition, the NN runtime for layer node i can be calculated as:

$$t_l(H, W, N_l[i]) = (2H + W + d_{1,i} - 2) \times \lceil d_{2,i}/N_l[i] \rceil / H \times \lceil d_{3,i}/W \rceil \quad (1)$$

where H and W are sub-array height and width. $d_{1,i}$, $d_{2,i}$ and $d_{3,i}$ are layer dimensions m, n, k . Assuming R_l is a set collecting all layer nodes within a loop, NN total runtime is:

$$t_{nn}(H, W, N_l) = \sum_i^{R_l} t_l(H, W, N_l[i]) \quad (2)$$

For a VSA node j , runtime for spatial and temporal mapping are respectively:

$$t_{v,spatial}(H, W, N_v[j]) = n_j \times \lceil d_j / (W \times H \times N_v[j]) \rceil \times T \quad (3)$$

$$t_{v,temp}(H, W, N_v[j]) = \lceil n_j / W \rceil \times \lceil d_j / H \times N_v[j] \rceil \times T \quad (4)$$

where $T = 3 \times H + d_j - 1$, n_j and d_j are the vector quantity and size respectively. DAG uses the fastest mapping scheme, so with R_v as the set of all VSA nodes, the total VSA runtime in a single loop is:

$$t_{vsa}(H, W, N_v) = \min(\sum_j^{R_v} t_{v,temp}(H, W, N_v[j]), \sum_j^{R_v} t_{v,spatial}(H, W, N_v[j])) \quad (5)$$

AdArray Design Generation. We describe how DAG generates AdArray design and mapping scheme in Algorithm 1. To mitigate the search space, the DSE process is decoupled into 2 phases, exploiting AdArray's two-level flexibility (Sec. IV-B):

In **Phase 1**, DAG assumes static partitions among nodes to limit search space (i.e. $\forall i, j, N_l[i] = \bar{N}_l, N_v[j] = \bar{N}_v$). It finds the optimal H, W, N constrained by maximum number of PEs M defined based on FPGA resource, and a fixed partition scheme defined by $\bar{N}_l$ and $\bar{N}_v$ to maximizes overall performance. The search space of H, W, N is also pruned based on analytical model results. **Phase 2** explores the

TABLE III
DESIGN CONFIGURATION AND FPGA DEPLOYMENT.

Workloads	Precision		AdArray Configuration		SIMD Size	On-chip SRAM Blocks (BRAM)				On-chip Cache (URAM)	AMD U250 Utilization						Frequency
	NN	Symb	Size (H, W, N)	Default Partition ($\bar{N}_l : \bar{N}_v$)		MemA1, MemA2	Mem B	Mem C	DSP		LUT	FF	BRAM	URAM	LUTRAM		
NVSA	INT8	INT4	32, 16, 16	14 : 2	64	2.7 MB, 1.1 MB	2.7 MB	1.6 MB	16.2 MB	89%	56%	60%	34%	8%	24%	272 MHz	
MIMONet	INT8	INT8	32, 32, 8	6 : 2	64	3.4 MB, 1.2 MB	3.4 MB	2.1 MB	20.1 MB	89%	44%	52%	43%	10%	20%	272 MHz	
LVRf	INT8	INT4	32, 16, 16	14 : 2	64	2.7 MB, 0.96 MB	2.7 MB	1.4 MB	15.5 MB	89%	56%	60%	31%	7%	24%	272 MHz	

TABLE IV

NSFLOW ALGORITHM OPTIMIZATION PERFORMANCE. NSFlow exhibits comparable reasoning capability with the proposed mixed precision.

Reasoning Accuracy	FP32	FP16	INT8	MP (IN8 for NN, INT4 for Symb)	INT4
RAVEN [32]	98.9%	98.9%	98.7%	98.0%	92.5%
I-RAVEN [33]	99.0%	98.9%	98.8%	98.1%	91.3%
PGM [34]	68.7%	68.6%	68.4%	67.4%	59.9%
Memory	32MB	16MB	8MB	5.5MB	4MB

mapping scheme further by efficiently fine-tuning N_l and N_v around $\bar{N}_l$ and $\bar{N}_v$ in the dataflow loop with maximum number of iterations pre-defined as $Iter_{max}$, seeking optimal or near-optimal partitions. Searching granularity is set to the span of each NN layer, as VSA kernels are in general smaller and more flexible to be fit into arbitrary array shapes. With $O(N)$ complexity, our two-phase DSE algorithm shrinks the design space by $10^{100} \times$ shown in Tab. IV.

Memory and SIMD unit. After generating the design of AdArray, memory blocks sizes are computed based on node memory cost to eliminate inner-node memory stalls, for example $M_{A1} = \max(filter\ size\ in\ R_l)$, $M_{A2} = \max(node\ size\ in\ R_v)$. M_{A1} and M_{A2} are merged for non-parallel operations. On-chip cache size is $2 \times (M_A + M_B + M_C)$. SIMD size is minimized such that latency of concurrent elem-wise/vector reduction operations can be hidden.

Unlike other DSE work [35], [36] that only focuses on single task mapping on traditional systolic array, NSFlow exploits NSAI inter-task and inner-task parallelism opportunities on AdArray at both hardware level and mapping level, boosting the performance of versatile NSAI workloads.

VI. EVALUATION RESULTS

A. Experimental Setup

Algorithm setup. We evaluate NSFlow with three state-of-the-art VSA-based NSAI workloads, i.e., NVSA [17], MIMONet [28], and LVRf [12] on the commonly-used spatial-temporal reasoning datasets - RAVEN [32], I-RAVEN [33], PGM [34], CVR [37], and SVRT [38]. Following [12], [17], [28], we select the training hyperparameters based on the end-to-end reasoning performance on the validation set.

Hardware setup. We consider several hardware baselines, including TX2, Xavier NX, Xeon CPU, RTX 2080, and ML accelerators (TPU, Xilinx DPU). NSFlow framework can be deployed on any type of FPGA board. Tab. III showcases our deployment for 3 algorithms on AMD U250 using Xilinx Vivado and Synopsys Design Compiler. The clock frequency is set to 272MHz.

B. NSFlow Performance

Mixed-precision performance. We benchmark NSAI model on three spatial-temporal reasoning datasets to first evaluate the effectiveness of mixed quantization in NSFlow. As shown in Tab. IV, we can observe that NSFlow mixed precision achieves comparable accuracy with NVSA algorithm [17] while with 5.8 $\times$ memory footprint savings. Similar results are observed in MIMONet/LVRf on CVR/SVRT datasets. It is also worth noting that neurosymbolic methods consistently achieve improved cognition and reasoning capability than neural network-based methods and surpass human performance.

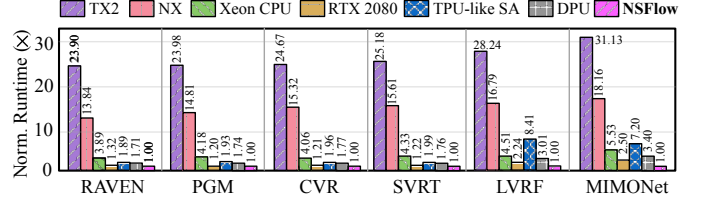


Fig. 5. **End-to-End Runtime Improvement.** NSFlow consistently outperforms Xilinx DPU, TPU-like accelerator, Xeon CPU, RTX GPU, and edge SoCs (TX2, NX) in end-to-end runtime evaluated on NSAI reasoning tasks.

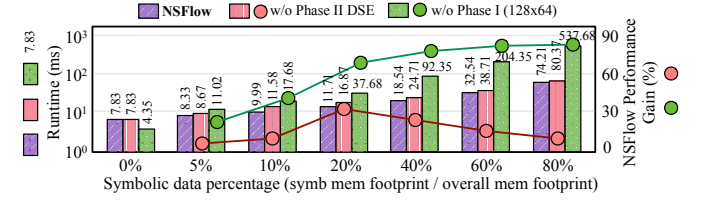


Fig. 6. **Ablation Study.** NSFlow exhibits superior scalability comparing to normal TPU design across workloads with various symbolic proportions.

Performance improvement. We first benchmark our NSFlow accelerator against edge SoC (Jetson TX2, Xavier NX), Intel Xeon CPU, Nvidia RTX 2080, TPU-like systolic array (128 $\times$ 128), and Xilinx DPU. For accelerating NSAI algorithm on six reasoning tasks featuring different difficulties. We can observe in Fig. 5 that NSFlow accelerator consistently outperforms other devices, offering 31 $\times$ /18 $\times$ speedup over TX2 and NX, more than 2 $\times$ over GPU, up to 8 $\times$ speedup over TPU-like systolic array, and more than 3 $\times$ speedup over Xilinx DPU on some standard workloads.

Ablation study. To further showcase the scalability of NSFlow and validate the necessity of proposed DSE algorithm, in Fig. 6 we summarize the runtime of an NSFlow-generated architecture (32 $\times$ 32 $\times$ 8) w/ and w/o the proposed mapping and hardware techniques, evaluated on a NVSA-like workload with varying vector-symbolic data proportions alongside a ResNet18. We observe that despite slight overhead caused by array partition when symbolic part is minimal (< 1%), (1) with symbolic ratio going up NSFlow speedup against traditional systolic array grows steadily, reaching up to more than 7 $\times$ when symbolic data occupies 80% of the memory. (2) The performance gain from our two-phase DSE algorithm (compared to only having array folding, or Phase I) can reach 44% when symbolic workload is balanced with NN (symbolic memory percentage $\approx$ 20%). These findings highlight NSFlow’s scalability to adapt to varying workloads and its efficiency in handling symbolic-heavy scenarios.

VII. CONCLUSION

To enable efficient NSAI for real-time cognitive applications, we propose NSFlow, the first end-to-end design automation framework to accelerate NSAI systems. NSFlow leverages the unique NSAI workload characteristics, explores dataflow and architecture design space, and generates scalable designs for FPGA deployment. We believe NSFlow paves the way for advancing efficient cognitive reasoning systems and unlocking new possibilities in NSAI.

REFERENCES

- [1] J. Mao, C. Gan, P. Kohli, J. B. Tenenbaum, and J. Wu, "The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision," *International Conference on Learning Representations (ICLR)*, 2019.
- [2] C. Han, J. Mao, C. Gan, J. Tenenbaum, and J. Wu, "Visual concept-metaconcept learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [3] L. Mei, J. Mao, Z. Wang, C. Gan, and J. B. Tenenbaum, "Falcon: fast visual concept learning by integrating images, linguistic descriptions, and conceptual relations," *International Conference on Learning Representations (ICLR)*, 2022.
- [4] K. Yi, C. Gan, Y. Li, P. Kohli, J. Wu, A. Torralba, and J. B. Tenenbaum, "Cleverer: Collision events for video representation and reasoning," in *International Conference on Learning Representations (ICLR)*, 2020.
- [5] C. Zhang, B. Jia, S.-C. Zhu, and Y. Zhu, "Abstract spatial-temporal reasoning via probabilistic abduction and execution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9736–9746, 2021.
- [6] V. Shah, A. Sharma, G. Shroff, L. Vig, T. Dash, and A. Srinivasan, "Knowledge-based analogical reasoning in neuro-symbolic latent spaces," *arXiv preprint arXiv:2209.08750*, 2022.
- [7] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, no. 7995, pp. 476–482, 2024.
- [8] M. Ibrahim, Z. Wan, H. Li, P. Panda, T. Krishna, P. Kanerva, Y. Chen, and A. Raychowdhury, "Special session: Neuro-symbolic architecture meets large language models: A memory-centric perspective," in *2024 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ ISSS)*, pp. 11–20, IEEE, 2024.
- [9] Z. Wan, C.-K. Liu, H. Yang, R. Raj, C. Li, H. You, Y. Fu, C. Wan, A. Samajdar, Y. C. Lin, *et al.*, "Towards cognitive ai systems: Workload and characterization of neuro-symbolic ai," in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 268–279, IEEE, 2024.
- [10] G. Booch, F. Fabiano, L. Horesh, K. Kate, J. Lenchner, N. Linck, A. Loreggia, K. Muresan, N. Mattei, F. Rossi, *et al.*, "Thinking fast and slow in ai," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 15042–15046, 2021.
- [11] G. Camposampiero, M. Hersche, A. Terzić, R. Wattenhofer, A. Sebastian, and A. Rahimi, "Towards learning abductive reasoning using vsa distributed representations," in *International Conference on Neural-Symbolic Learning and Reasoning*, pp. 370–385, Springer, 2024.
- [12] M. Hersche, F. Di Stefano, T. Hofmann, A. Sebastian, and A. Rahimi, "Probabilistic abduction for visual abstract reasoning via learning rules in vector-symbolic architectures," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [13] P. Hitzler, A. Eberhart, M. Ebrahimi, M. K. Sarker, and L. Zhou, "Neuro-symbolic approaches in artificial intelligence," *National Science Review*, vol. 9, no. 6, p. nwac035, 2022.
- [14] K. Hamilton, A. Nayak, B. Božić, and L. Longo, "Is neuro-symbolic ai meeting its promises in natural language processing? a structured review," *Semantic Web*, vol. 15, no. 4, pp. 1265–1306, 2024.
- [15] Z. Wan, C.-K. Liu, H. Yang, R. Raj, C. Li, H. You, Y. Fu, C. Wan, S. Li, Y. Kim, *et al.*, "Towards efficient neuro-symbolic ai: From workload characterization to hardware architecture," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024.
- [16] Z. Wan, C.-K. Liu, M. Ibrahim, H. Yang, S. Spetalnick, T. Krishna, and A. Raychowdhury, "H3dfact: Heterogeneous 3d integrated cim for factorization with holographic perceptual representations," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, IEEE, 2024.
- [17] M. Hersche, M. Zeqiri, L. Benini, A. Sebastian, and A. Rahimi, "A neuro-vector-symbolic architecture for solving raven's progressive matrices," *Nature Machine Intelligence*, vol. 5, no. 4, pp. 363–375, 2023.
- [18] Z. Wan, C.-K. Liu, H. Yang, C. Li, H. You, Y. Fu, C. Wan, T. Krishna, Y. Lin, and A. Raychowdhury, "Towards cognitive ai systems: a survey and prospective on neuro-symbolic ai," *arXiv preprint arXiv:2401.01040*, 2024.
- [19] J. Wang, L. Guo, and J. Cong, "Autosa: A polyhedral compiler for high-performance systolic arrays on fpga," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 93–104, 2021.
- [20] X. Wei, C. H. Yu, P. Zhang, Y. Chen, Y. Wang, H. Hu, Y. Liang, and J. Cong, "Automated systolic array architecture synthesis for high throughput cnn inference on fpgas," in *Proceedings of the 54th Annual Design Automation Conference 2017*, pp. 1–6, 2017.
- [21] Z. Li, Y. Zhang, J. Wang, and J. Lai, "A survey of fpga design for ai era," *Journal of Semiconductors*, vol. 41, no. 2, p. 021402, 2020.
- [22] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, "Understanding the potential of fpga-based spatial acceleration for large language model inference," *ACM Transactions on Reconfigurable Technology and Systems*, 2024.
- [23] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, "A comprehensive evaluation of fpga-based spatial acceleration of llms," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pp. 185–185, 2024.
- [24] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, *et al.*, "Flightllm: Efficient large language model inference with a complete mapping flow on fpgas," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pp. 223–234, 2024.
- [25] W. Huang, H. Wu, Q. Chen, C. Luo, S. Zeng, T. Li, and Y. Huang, "Fpga-based high-throughput cnn hardware accelerator with high computing resource utilization ratio," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 8, pp. 4069–4083, 2021.
- [26] S. Yin, S. Tang, X. Lin, P. Ouyang, F. Tu, L. Liu, and S. Wei, "A high throughput acceleration for hybrid neural networks with efficient resource management on fpga," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 4, pp. 678–691, 2018.
- [27] H. Xu, Y. Li, and S. Ji, "Llamaf: An efficient llama2 architecture accelerator on embedded fpgas," *arXiv preprint arXiv:2409.11424*, 2024.
- [28] N. Menet, M. Hersche, G. Karunaratne, L. Benini, A. Sebastian, and A. Rahimi, "Mimonets: Multiple-input-multiple-output neural networks exploiting computation in superposition," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [29] Z. Wan, H. Yang, R. Raj, C.-K. Liu, A. Samajdar, A. Raychowdhury, and T. Krishna, "Cogsys: Efficient and scalable neurosymbolic cognition system via algorithm-hardware co-design," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 775–789, IEEE, 2025.
- [30] M. Langhammer, S. Gribok, and G. Baekler, "High density 8-bit multiplier systolic arrays for fpga," in *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 84–92, IEEE, 2020.
- [31] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of dnn accelerators using scale-sim," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 58–68, IEEE, 2020.
- [32] C. Zhang, F. Gao, B. Jia, Y. Zhu, and S.-C. Zhu, "Raven: A dataset for relational and analogical visual reasoning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 5317–5327, 2019.
- [33] S. Hu, Y. Ma, X. Liu, Y. Wei, and S. Bai, "Stratified rule-aware network for abstract visual reasoning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 1567–1574, 2021.
- [34] D. Barrett, F. Hill, A. Santoro, A. Morcos, and T. Lillicrap, "Measuring abstract reasoning in neural networks," in *International conference on machine learning (ICML)*, pp. 511–520, PMLR, 2018.
- [35] S.-C. Kao and T. Krishna, "Gamma: Automating the hw mapping of dnn models on accelerators via genetic algorithm," in *Proceedings of the 39th International Conference on Computer-Aided Design*, pp. 1–9, 2020.
- [36] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings," *IEEE micro*, vol. 40, no. 3, pp. 20–29, 2020.
- [37] A. Zerroug, M. Vaishnav, J. Colin, S. Musslick, and T. Serre, "A benchmark for compositional visual reasoning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 29776–29788, 2022.
- [38] F. Fleuret, T. Li, C. Dubout, E. K. Wampler, S. Yantis, and D. Geman, "Comparing machines and humans on a visual categorization test," *Proceedings of the National Academy of Sciences*, vol. 108, no. 43, pp. 17621–17625, 2011.